Tomghost: Write Up

Plataforma: TryHackMe



Máquina / Desafio:	Tomghost
Dificuldade:	Fácil
Data de Resolução:	27/10/2026
Autor:	Alan Dias (alankk)

27 de outubro de 2026

Sumário

1 Visão Geral **2**

2 Reconhecimento **2**

3 Exploração **3**

4 Movimentação Lateral **5**

5 Escalada de Privilégios **6**

6 Conclusão **7**

7 Referências **8**

1 Visão Geral

A máquina **Tomghost**, hospedada na plataforma TryHackMe, apresenta um cenário prático voltado à identificação e exploração de vulnerabilidades em servidores de aplicação Java e conectores de protocolo interno, associada à quebra de artefatos criptográficos e ao abuso de binários privilegiados no sistema operacional Linux. A superfície de ataque exposta engloba serviços de gerenciamento remoto e um servidor web executando sobre um conector de integração com protocolo legado. O percurso de intrusão compreende o mapeamento de serviços expostos para a exploração de leitura não autenticada de arquivos internos na aplicação web, viabilizando a obtenção de credenciais de acesso para estabelecimento do shell inicial via SSH. Em seguida, a etapa de pós-exploração fundamenta-se na análise e quebra de chaves criptográficas para a realização de movimentação lateral entre contas de usuário e, por fim, na identificação de diretivas permissivas de execução no Sudo para a consolidação de privilégios como `root`.

2 Reconhecimento

O mapeamento da superfície de ataque iniciou-se por meio de uma varredura de portas TCP com o utilitário Nmap, visando detectar portas abertas, serviços ativos e versões operacionais no endereço de rede do alvo, conforme demonstrado no comando a seguir:

```
nmap -sC -T4 -sV --min-rate 5000 <TARGET_IP>
```

Listing 1: Varredura de portas e serviços via Nmap

O relatório da varredura, ilustrado na Figura 1, retornou as seguintes portas TCP em estado aberto:

- **Porta 22/tcp (SSH):** Serviço OpenSSH versão 7.2p2 executando sobre o sistema operacional Ubuntu (protocolo 2.0).
- **Porta 53/tcp (DNS):** Serviço identificado como `tcpwrapped`.
- **Porta 8009/tcp (AJP13):** Conector Apache JServ Protocol versão 1.3 ativo com suporte aos métodos `GET`, `HEAD`, `POST` e `OPTIONS`.
- **Porta 8080/tcp (HTTP):** Servidor web Apache Tomcat na versão 9.0.30.

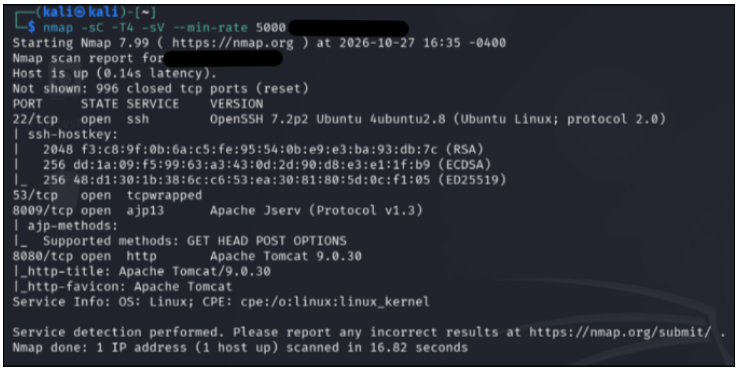


Figura 1: Resultado da varredura de portas e serviços via Nmap.

A correlação entre o Apache Tomcat versão 9.0.30 na porta 8080 e a presença do conector AJP13 exposto na porta 8009 indicou um vetor propício para a exploração de falhas intrínsecas ao conector de integração do Tomcat.

3 Exploração

A análise de vulnerabilidades associadas à versão identificada do Apache Tomcat (9.0.30) e à exposição do protocolo AJP revelou a suscetibilidade à falha crítica catalogada sob o boletim **CVE-2020-1938**, amplamente conhecida como **Ghostcat** [1].

A falha decorre de uma deficiência estrutural no conector AJP (Apache JServ Protocol), habilitado por padrão em versões legadas do Tomcat. O protocolo confia implicitamente nos atributos repassados pelo cliente sem autenticação prévia. Ao enviar requisições AJP especialmente forjadas manipulando atributos internos do contêiner servlet (como `javax.servlet.include.request_uri` e `javax.servlet.include.servlet_path`), um atacante remoto não autenticado consegue instruir o servlet padrão a ler o conteúdo bruto de arquivos arbitrários localizados no diretório da aplicação web (`webapps`), contornando as proteções tradicionais que impedem o acesso a pastas internas como a `WEB-INF`.

Para validar a vulnerabilidade e inspecionar os arquivos de configuração do contêiner, utilizou-se o script de prova de conceito em Python [2], direcionando a varredura contra a porta AJP 8009 do alvo:

```
python3 poc_debug.py -t <TARGET_IP> -w -o results/
```

Listing 2: Execução do exploit Ghostcat contra o conector AJP

A Figura 2 comprova a execução bem-sucedida do exploit, que extraiu o arquivo de implantação da aplicação web (`/WEB-INF/web.xml`) com tamanho de 1261 bytes.



Figura 2: Execução da PoC para a falha Ghostcat e leitura de /WEB-INF/web.xml.

Ao inspecionar o arquivo `web.xml` recuperado, constatou-se a presença de credenciais administrativas documentadas indevidamente na tag descritiva da aplicação, conforme apresentado a seguir:

```
<display-name>Welcome to Tomcat</display-name>
<description>
  Welcome to GhostCat
  skyfuck <PASSWORD>
</description>
```

Listing 3: Trecho extraído do arquivo web.xml contendo credenciais

A Figura 3 detalha a exibição das credenciais em texto claro associadas à conta de usuário `skyfuck`.



Figura 3: Credenciais em texto claro identificadas no arquivo web.xml.

Com o par de credenciais obtido (`skyfuck:<PASSWORD>`), realizou-se a conexão direta ao serviço OpenSSH ativo na porta 22:

```
ssh skyfuck@<TARGET_IP>
cd /home/merlin/
cat user.txt
```

Listing 4: Autenticação SSH e leitura da flag de usuário

A autenticação foi realizada com sucesso, consolidando o acesso inicial (foothold) ao sistema sob o usuário `skyfuck`. Navegando até o diretório pessoal do usuário `merlin` (`/home/merlin`), constatou-se que as permissões do sistema permitiam a leitura do arquivo `user.txt`, obtendo a flag de usuário (`<FLAG>`), conforme registrado na Figura 4.

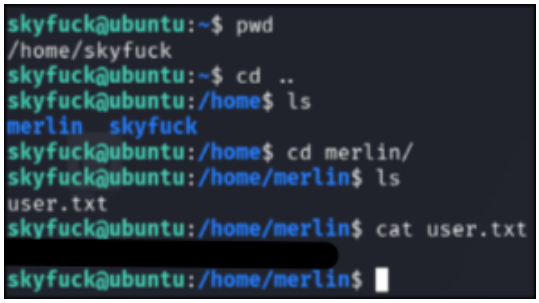


Figura 4: Acesso inicial via SSH como skyfuck e captura da flag de usuário.

4 Movimentação Lateral

Após a captura da primeira flag, iniciou-se a enumeração do diretório pessoal do usuário `skyfuck` (`/home/skyfuck`). Identificaram-se dois artefatos criptográficos sensíveis armazenados no local:

- `tryhackme.asc`: Bloco de chave privada PGP/GnuPG em formato ASCII blindado.
- `credential.pgp`: Arquivo binário cifrado contendo informações confidenciais.

A Figura 5 documenta a presença dos arquivos e a visualização do cabeçalho da chave privada PGP.

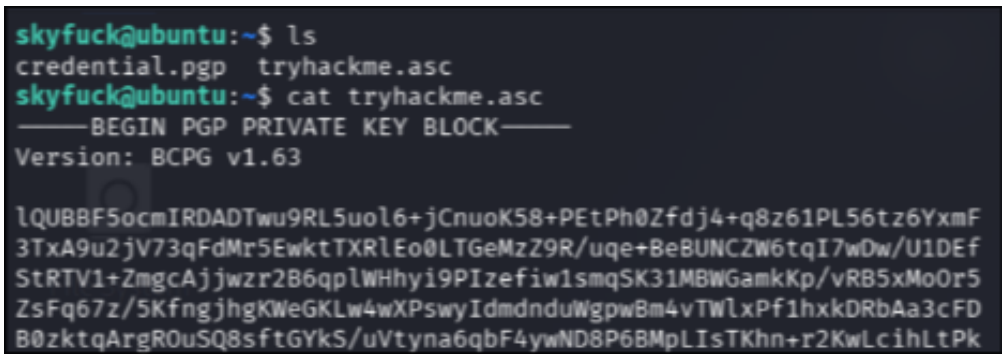


Figura 5: Localização dos arquivos credential.pgp e tryhackme.asc no diretório home.

Para possibilitar o uso da chave privada na decriptação dos dados, fez-se necessário descobrir a frase secreta (passphrase) que a protegia. Transferiu-se a chave privada `tryhackme.asc` para a máquina de ataque e utilizou-se o utilitário `gpg2john` para extrair o hash da frase secreta em um formato compatível com o John the Ripper:

```
gpg2john tryhackme.asc > thm.txt
john --wordlist=/usr/share/wordlists/rockyou.txt thm.txt
```

Listing 5: Extração de hash e quebra de senha da chave PGP

O ataque baseado no dicionário `rockyou.txt` foi concluído instantaneamente, revelando a frase secreta associada à chave privada (`<PASSWORD>`), como exibido na Figura 6.

Retornando à sessão no alvo, importou-se a chave privada no chaveiro local do GPG e procedeu-se com a decriptação do arquivo `credential.pgp`, fornecendo a frase secreta recuperada:

```
export GPG_TTY=$(tty)
gpg --import tryhackme.asc
gpg --decrypt credential.pgp
```

Listing 6: Importação da chave PGP e decriptação da credencial

A Figura 7 demonstra o resultado da decriptação, que expôs a senha de acesso da conta de usuário `merlin` (`merlin:<PASSWORD>`).

Com as novas credenciais, consolidou-se a movimentação lateral autenticando-se via SSH diretamente na conta do usuário `merlin`:

```
ssh merlin@<TARGET_IP>
```

Listing 7: Autenticação SSH na conta merlin

A transição horizontal de usuário foi concluída com sucesso, viabilizando o acesso ao novo contexto de permissões do sistema.

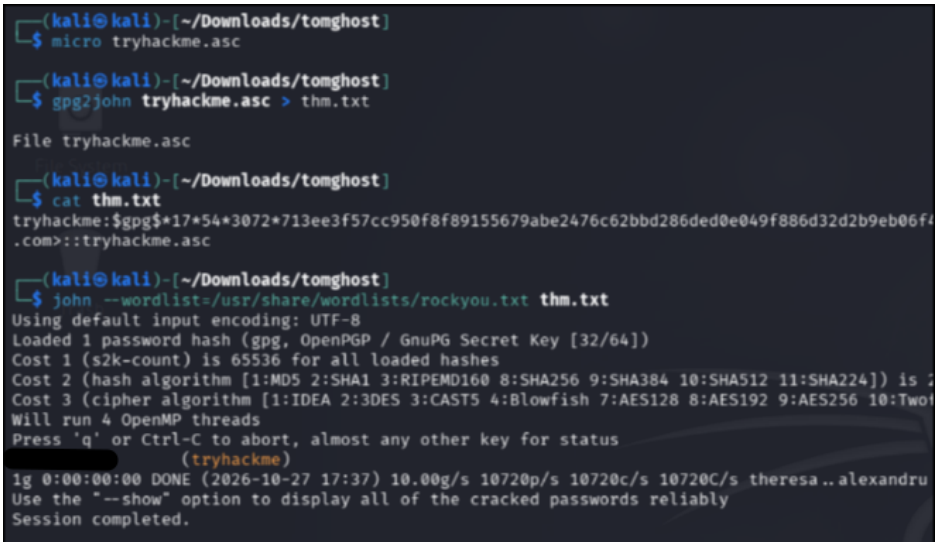


Figura 6: Extração de hash com gpg2john e quebra da frase secreta via John the Ripper.

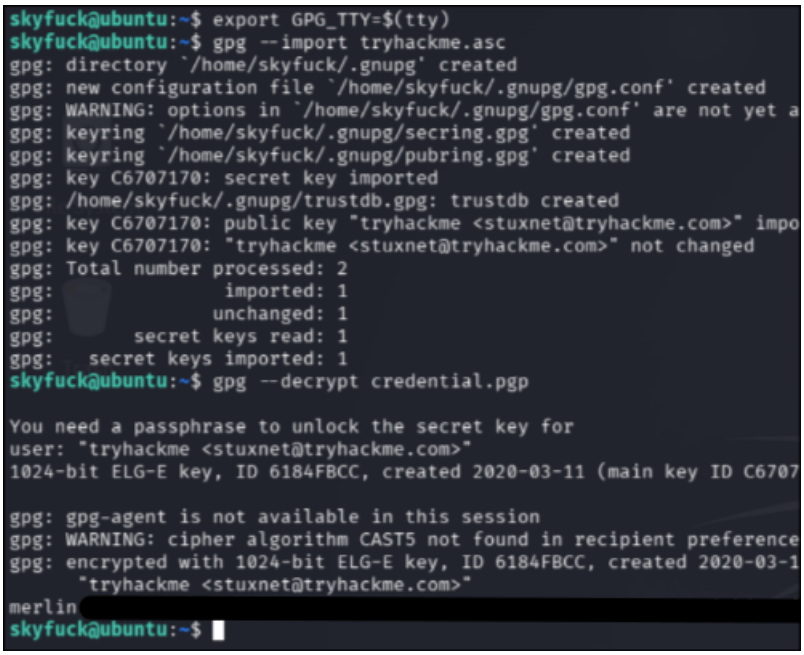


Figura 7: Importação da chave PGP e decriptação da senha do usuário merlin.

5 Escalada de Privilégios

Sob o contexto da conta `merlin`, procedeu-se com a enumeração local de privilégios executando a verificação de comandos autorizados no Sudoers:

```
sudo -l
```

Listing 8: Enumeração de permissões no sudoers para o usuário merlin

A saída do comando revelou uma diretiva de concessão de privilégios sem exigência de senha (`NOPASSWD`) para o utilitário de compressão de arquivos `/usr/bin/zip`:

```
User merlin may run the following commands on ubuntu:
    (root : root) NOPASSWD: /usr/bin/zip
```

Listing 9: Diretiva de execução identificada no sudoers

A Figura 8 evidencia o login na conta `merlin` e a listagem da regra permissiva no Sudo.

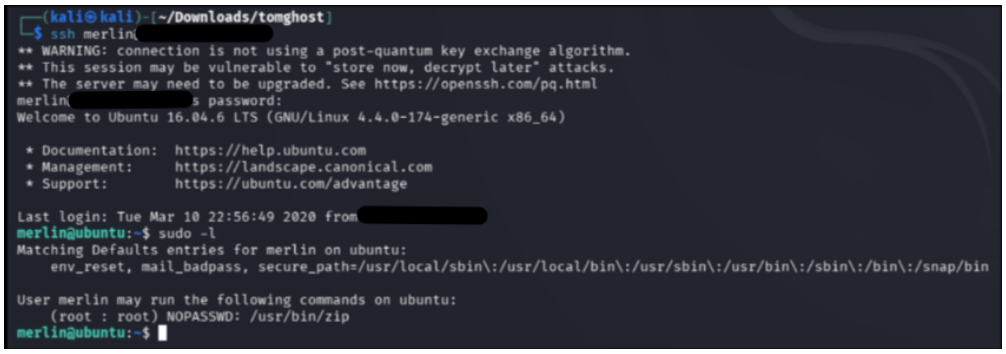


Figura 8: Sessão SSH como merlin e enumeração de regras no sudoers.

De acordo com a base de conhecimento GTFOBins [3], o utilitário `zip` suporta os parâmetros `-T` (testagem de integridade de arquivos) e `-TT` (especificação do comando de descompressão a ser utilizado no teste). Quando invocado com privilégios de superusuário, o binário permite a execução arbitrária de comandos sob o contexto de `root`.

Executou-se a linha de comando de elevação invocando o interpretador de comandos `/bin/sh`:

```
sudo zip /tmp/temp.zip /etc/hosts -T -TT '/bin/sh #'
whoami
cat /root/root.txt
```

Listing 10: Escape de privilégios via sudo zip e captura da flag final

O utilitário executou a instrução sob o identificador UID 0 (`root`), gerando um shell administrativo interativo. Confirmou-se o nível de privilégio máximo através do comando `whoami` e, em seguida, realizou-se a leitura do arquivo `/root/root.txt`, capturando a flag de root (`<FLAG>`), como demonstrado na Figura 9.

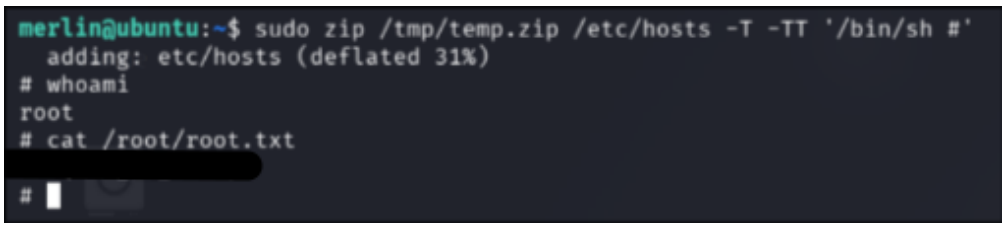


Figura 9: Execução de comando via sudo zip, obtenção de root e leitura da flag final.

6 Conclusão

A resolução da máquina Tomghost demonstrou uma cadeia de comprometimento consistente baseada em falhas de configuração de serviços legados e má gestão de segredos locais. A intrusão inicial explorou a

vulnerabilidade Ghostcat (CVE-2020-1938) no conector AJP do Apache Tomcat, viabilizando a leitura de credenciais em texto claro no arquivo `web.xml`. A movimentação lateral decorreu da quebra da frase secreta de uma chave privada PGP e posterior decriptação das credenciais do usuário Merlin. Por fim, a delegação indevida de execução com privilégios de superusuário sem senha para o utilitário Zip no Sudoers propiciou o escape e a obtenção de controle total do host como root.

7 Referências

Referências

[1] **National Vulnerability Database (NVD)**: CVE-2020-1938 - Apache Tomcat AJP File Read/Inclusion (Ghostcat).
Disponível em: https://nvd.nist.gov/vuln/detail/CVE-2020-1938

[2] **Repositório de Exploit Público**: Apache Tomcat AJP Ghostcat Exploit PoC.
Disponível em: https://github.com/abrewer251/CVE-2020-1938_Ghostcat-PoC

[3] **GTFOBins**: Zip Privilege Escalation and Shell Spawning Techniques.
Disponível em: https://gtfobins.github.io/gtfobins/zip/

[4] **TryHackMe**: Sala Tomghost.
Disponível em: https://tryhackme.com/room/tomghost